

Project: Open Port Scan and Service Mapping

Tools & Methodology Documentation DEV-353

Category: Category: Network Security / Network Reconnaissance

1. Objective

The primary objective of this project was to perform systematic TCP port discovery and service mapping across a defined set of authorized target systems.

The assessment was designed to:

- Identify open TCP ports across the complete port range.
- Determine the services exposed through discovered open ports.
- Identify service and version information where available.
- Automate the scanning process across multiple targets.
- Preserve raw scan results for verification and analysis.

The methodology was designed as a repeatable two-stage scanning process:

Full TCP Port Discovery → Service & Version Detection

2. Tools & Technologies Used

The project uses a lightweight toolchain consisting primarily of **Nmap**, **Ubuntu/Linux**, and **Bash automation**.

Category	Tool / Technology	Version	Purpose
Operating System	Ubuntu/Linux	—	Scanning and execution environment

Network Scanner	Nmap	7.98	TCP port and service discovery
Automation	Bash	—	Multi-target scan automation
Network Protocol	TCP/IP	—	Port-level network communication
Version Control	Git	—	Project version management
Repository	GitHub	—	Source code and documentation hosting
Documentation	Markdown	—	Technical project documentation

2.1 Nmap

Nmap 7.98 is the primary tool used to discover open TCP ports and identify services and versions.

- **Port Discovery**

`nmap -Pn -p- "$IP"`

Scans all TCP ports from **1–65535** without relying on host discovery.

- **Service Detection**

`nmap -Pn -sV -p "$OPEN_PORTS" "$IP"`

Identifies services and available version information on the discovered open ports.

2.2 Ubuntu/Linux

Ubuntu/Linux is used as the environment for running Nmap, Bash scripts, and managing project files.

2.3 Bash

Bash automates target validation, scanning, open-port extraction, service detection, and result storage.

`MAX_PARALLEL=4`

Limits concurrent target processing to a maximum of **four targets**.

2.4 `targets.txt`

Stores the **15 authorized IPv4 addresses** used as the scanning input.

2.5 `scan_all.sh`

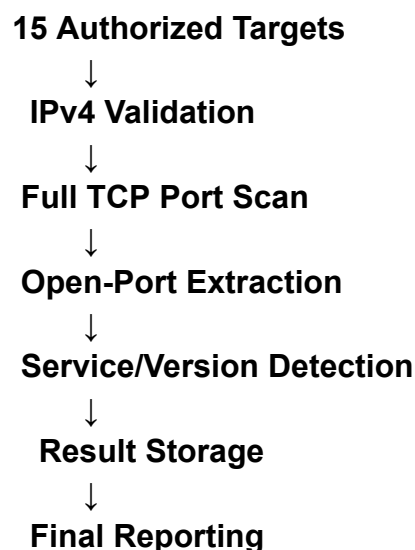
The main Bash script that automates the complete scanning process for all targets.

2.6 Git/GitHub

Git and GitHub are used to maintain project code, documentation, scan outputs, and reports under version control.

3. Scanning Methodology

The project uses a **two-stage TCP scanning process**:



4. Phase 1 — Target Preparation

The authorized IP addresses are stored in `targets.txt` and provided to `scan_all.sh` as the scanning scope.

5. Phase 2 — Target Validation

The script validates each target as an IPv4 address before starting the Nmap scan.

Target → IPv4 Validation → Valid → Scan → Invalid → Reject

6. Phase 3 — Full TCP Port Discovery

```
nmap -Pn -p- "$IP"
```

Scans the complete TCP port range **1–65535** to identify open ports.

7. Phase 4 — Open-Port Identification

The script extracts ports reported as **open** from the initial Nmap results and prepares them for service detection.

Example:

22/tcp

80/tcp

443/tcp

8. Phase 5 — Service and Version Detection

```
nmap -Pn -sV -p "$OPEN_PORTS" "$IP"
```

Performs service and version detection only on the ports identified as open.

9. Nmap Parameters Used

Parameter	Purpose
<code>-Pn</code>	Scans targets without relying on host discovery.
<code>-p-</code>	Scans all TCP ports from 1 to 65535.
<code>-sV</code>	Detects services and available version information.

-p "\$OPEN_PORTS"	Scans only the ports discovered as open.
----------------------	--

10. Parameter Selection Rationale

The configuration directly supports the project's objective of **complete TCP port discovery followed by service mapping**.

Requirement	Configuration
Complete TCP scan	-p-
Avoid host-discovery dependency	-Pn
Service detection	-sV
Focus service scanning	Discovered open ports
Multiple-target automation	Bash
Controlled concurrency	MAX_PARALLEL=4

11. Multi-Target Automation

`scan_all.sh` applies the same scanning workflow to all **15 authorized targets**, providing consistent and repeatable execution.

12. Parallel Execution

MAX_PARALLEL=4

Allows up to **four targets to be processed concurrently** while keeping execution controlled.

13. Output Management

Scan results are organized into individual scan files and consolidated project reports.

```
scans/
├── <target>.nmap
├── <target>.gnmap
├── <target>.xml
└── service-detection.txt
```

```
reports/
├── final-report.md
└── raw-results.txt
```

14. Scan Output Structure

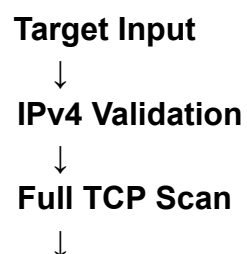
Output	Purpose
<code>.nmap</code>	Human-readable scan results
<code>.gnmap</code>	Grepable scan results
<code>.xml</code>	Structured Nmap results
<code>service-detection.txt</code>	Service/version results
<code>raw-results.txt</code>	Consolidated scan results
<code>final-report.md</code>	Final project report

15. Execution Workflow

The complete scanning process is executed through:

`./scan_all.sh`

Workflow:



Open-Port Extraction



Service Detection



Result Storage

16. Reproducibility

The project can be reproduced using the same target file, Bash script, and Nmap configuration.

```
chmod +x scan_all.sh
./scan_all.sh
```

Required environment:

- Ubuntu/Linux
- Nmap 7.98
- Bash
- Authorized IPv4 targets

17. Assessment Configuration Summary

Configuration	Project Setting
Project	Open Port Scan and Service Mapping
Targets	15 authorized IPv4 addresses
Primary Tool	Nmap 7.98
Execution Environment	Ubuntu/Linux

Automation	Bash
Target Input	<code>targets.txt</code>
Automation Script	<code>scan_all.sh</code>
Port Scope	TCP 1–65535
Host Handling	<code>-Pn</code>
Port Discovery	<code>-p-</code>
Service Detection	<code>-sV</code>
Service Scan Scope	Previously discovered open ports
Parallel Execution	Maximum 4 targets
Raw Output	<code>.nmap, .gnmap, .xml</code>
Service Output	<code>service-detection.txt</code>

Project Reports	<code>final-report.md, raw-results.txt</code>
------------------------	---

18. Methodology Summary

The project uses **Nmap 7.98 with Bash automation** to scan 15 authorized IPv4 targets. The workflow performs full TCP port discovery, extracts open ports, detects available services and versions, and stores the results for final reporting.